

Persistence and Persistence Rule Types:

Persistence methods are rules that identify traffic that should not be load-balanced but should be forwarded to the backend server that has processed requests from that source before.

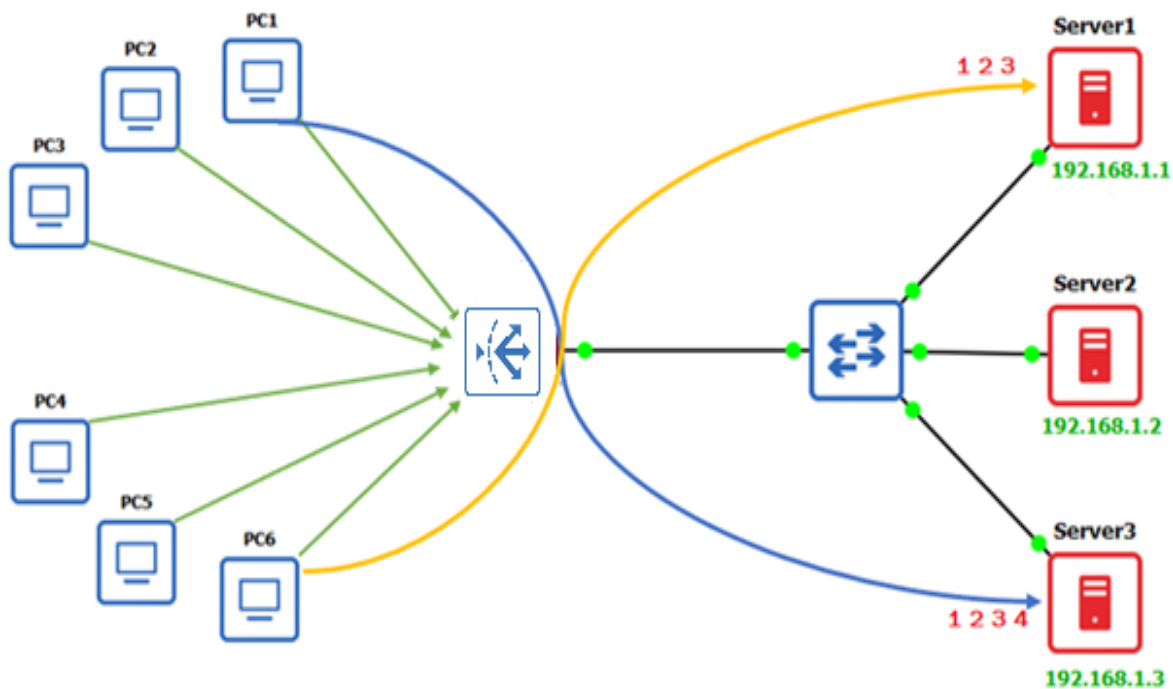
Persistence rules make sure that certain traffic always goes to the same back-end server instead of being shared among different servers. This is useful when a server needs to remember a user's previous requests, like in online shopping or voice calls, where maintaining a session is important.

FortiADC keeps track of persistence sessions to ensure client traffic is sent to the correct back-end server based on matching rules. It includes built-in persistence rules for easy setup and allows users to create custom rules based on different needs.

Persistence rules in FortiADC are used to ensure that traffic from a particular client is consistently directed to the same backend server. This is crucial for applications that require an established client-server session, such as e-commerce transactions or SIP voice calls.

Purpose:

Persistence rules help maintain session continuity by ensuring that requests from the same client are always sent to the same server.



Persistence Rule Types:

Different types of persistence rules give you flexibility in selecting the best option for your setup. Most rules also have timeout settings, allowing you to control how long a session remains active. FortiADC offers several persistence rule types to ensure that client requests are consistently routed to the same backend server during a session.

Source IP Address (LB_PERSIS_SRC_ADDR):

Persistence is based on the client's source IP address or subnet. This is the simplest form of persistence and is commonly used for web applications.

Source IP Hash (LB_PERSIS_HASH_SRC_ADDR):

Uses a hash of the client's source IP address to determine the server. This method provides a more even distribution of traffic compared to Source IP Address.

Source IP and Port Hash (LB_PERSIS_HASH_SRC_ADDR_PORT):

Combines the client's source IP address and port to determine the server. This is useful for applications that use multiple ports.

Cookie Hash (LB_PERSIS_HASH_COOKIE):

Uses a hash of a cookie provided by the client request. This is often used for web applications that set cookies for session management.

Address-Port hash:

Persistence is based on a hash of both the source IP address and TCP/UDP port number.

HTTP Header Hash:

Persistence is based on a hash of the HTTP header.

HTTP Request Hash:

Persistence is based on a hash of the specified URL parameters in the initial client request.

Cookie Hash:

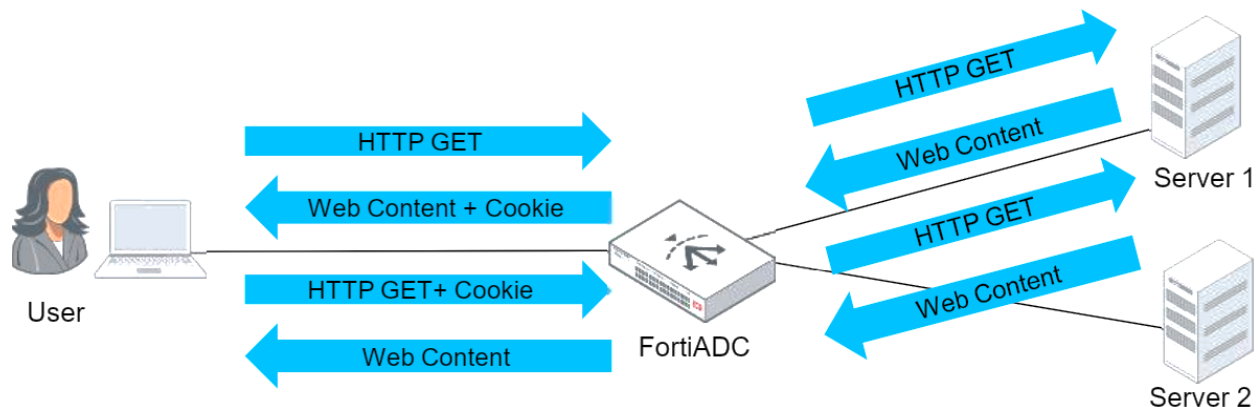
Persistence is based on a hash of the cookie provided by the server. The persistence can now be based on a field of the cookie, instead of the entire cookie.

Persistent Cookie:

The persistent cookie method is similar to the insert cookie method, but if the real server produces a cookie with the same name, then FortiADC won't modify it. Like the insert cookie method, the persistent cookie method also supports specifying a session time out.

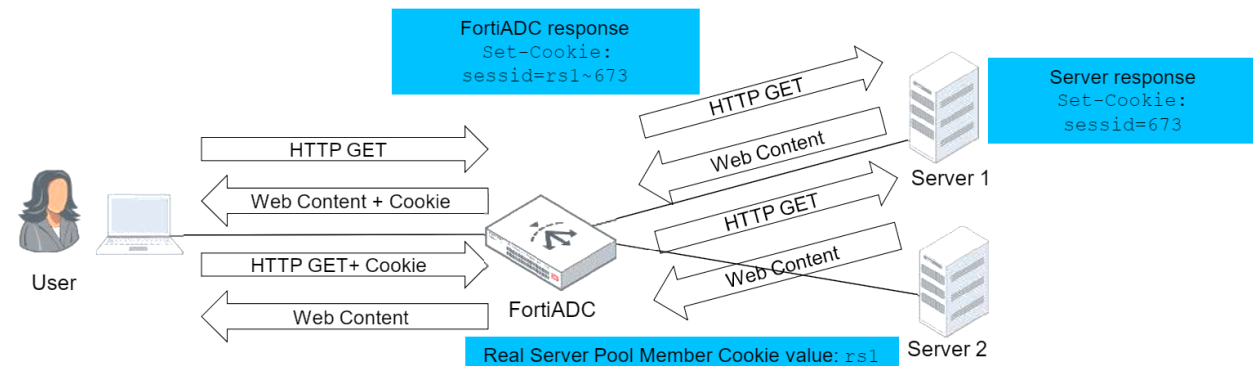
Insert Cookie:

The insert cookie rule uses the browser's cookie storage to maintain persistence. When a user connects for the first time, FortiADC selects a server using load balancing and forwards the request. The server responds with web content, and FortiADC adds a cookie to the response. After that, every time the user sends a request, the browser includes the cookie, helping FortiADC send the request to the same server. You can also set the cookie's domain value as a wildcard to allow cross-site usage.



Embedded Cookie:

The embedded cookie rule works by tracking a specific cookie sent by the server. When FortiADC gets a response from the server, it looks for that cookie and adds the server's ID as a prefix. After that, when the client sends requests, the cookie includes the server ID, allowing FortiADC to forward traffic to the correct server.



Rewrite Cookie:

Using this rule type, the cookie is provided by the real server, and FortiADC modifies its value.

RDP cookie:

Persistence is based on the RDP cookie sent by the RDP clients in the initial connection request.